It compares the current value of number (3) to 1. `3 > 1` so the while loop continues.

What is the next line?

Since the while loop condition was true the next line is: `print(".",end=" ")`

And it does what?

A third dot is printed on the line.

What is the next line?

It is: `number = number - 1`

What does it do?

It takes the current value of number (3) subtracts from it 1 and makes the 2 the new value of number.

What is the next line?

Back up to the start of the while loop: `while number > 1:`

What does it do?

It compares the current value of number (2) to 1. Since `2 > 1` the while loop continues.

What is the next line?

Since the while loop is continuing: `print(".",end=" ")`

What does it do?

It discovers the meaning of life, the universe and everything. I'm joking. (I had to make sure you were awake.) The line prints a fourth dot on the screen.

What is the next line?

It's: `number = number - 1`

What does it do?

Takes the current value of number (2) subtracts 1 and makes 1 the new value of number.

What is the next line?

Back up to the while loop: `while number > 1:`

What does the line do?

It compares the current value of number (1) to 1. Since `1 > 1` is false (one is not greater than one), the while loop exits.

What is the next line?

Since the while loop condition was false the next line is the line after the while loop exits, or: `print()`

What does that line do?

Makes the screen go to the next line.

Why doesn't the program print 5 dots?

The loop exits 1 dot too soon.

How can we fix that?

Make the loop exit 1 dot later.

And how do we do that?

There are several ways. One way would be to change the while loop to: `while number > 0:` Another way would be to change the conditional to: `number >= 1` There are a couple others.

7.0.4 How do I fix my program?

You need to figure out what the program is doing. You need to figure out what the program should do. Figure out what the difference between the two is. Debugging is a skill that has to be practiced to be learned. If you can't figure it out after an hour, take a break, talk to someone about the problem or contemplate the lint in your navel. Come back in a while and you will probably have new ideas about the problem. Good luck.

8 Defining Functions

8.0.1 Creating Functions

To start off this chapter I am going to give you an example of what you could do but shouldn't (so don't type it in):

```
a = 23
b = -23

if a < 0:
    a = -a
if b < 0:
    b = -b
if a == b:
    print("The absolute values of", a, "and", b, "are equal.")
else:
    print("The absolute values of", a, "and", b, "are different.")
```

with the output being:

```
The absolute values of 23 and 23 are equal.
```

The program seems a little repetitive. Programmers hate to repeat things – that's what computers are for, after all! (Note also that finding the absolute value changed the value of the variable, which is why it is printing out 23, and not -23 in the output.) Fortunately Python allows you to create functions to remove duplication. Here is the rewritten example:

```
a = 23
b = -23

def absolute_value(n):
    if n < 0:
        n = -n
    return n

if absolute_value(a) == absolute_value(b):
    print("The absolute values of", a, "and", b, "are equal.")
else:
    print("The absolute values of", a, "and", b, "are different.")
```

with the output being:

```
The absolute values of 23 and -23 are equal.
```

The key feature of this program is the `def` statement. `def`(short for define) starts a function definition. `def` is followed by the name of the function `absolute_value`. Next comes a '(' followed by the parameter `n` (`n` is passed from the program into the function when the

function is called). The statements after the ':' are executed when the function is used. The statements continue until either the indented statements end or a `return` is encountered. The `return` statement returns a value back to the place where the function was called. We already have encountered a function in our very first program, the `print` function. Now we can make new functions.

Notice how the values of `a` and `b` are not changed. Functions can be used to repeat tasks that don't return values. Here are some examples:

```
def hello():
    print("Hello")

def area(width, height):
    return width * height

def print_welcome(name):
    print("Welcome", name)

hello()
hello()

print_welcome("Fred")
w = 4
h = 5
print("width =", w, " height =", h, " area =", area(w, h))
```

with output being:

```
Hello
Hello
Welcome Fred
width = 4  height = 5  area = 20
```

That example shows some more stuff that you can do with functions. Notice that you can use no arguments or two or more. Notice also when a function doesn't need to send back a value, a return is optional.

8.0.2 Variables in functions

When eliminating repeated code, you often have variables in the repeated code. In Python, these are dealt with in a special way. So far all variables we have seen are global variables. Functions have a special type of variable called local variables. These variables only exist while the function is running. When a local variable has the same name as another variable (such as a global variable), the local variable hides the other. Sound confusing? Well, these next examples (which are a bit contrived) should help clear things up.

```
a = 4

def print_func():
    a = 17
    print("in print_func a =", a)

print_func()
print("a = ", a)
```

When run, we will receive an output of:

```
in print_func a = 17
a = 4
```

Variable assignments inside a function do not override global variables, they exist only inside the function. Even though `a` was assigned a new value inside the function, this newly assigned value was only relevant to `print_func`, when the function finishes running, and the `a`'s values is printed again, we see the originally assigned values.

Here is another more complex example.

```
a_var = 10
b_var = 15
e_var = 25

def a_func(a_var):
    print("in a_func a_var =", a_var)
    b_var = 100 + a_var
    d_var = 2 * a_var
    print("in a_func b_var =", b_var)
    print("in a_func d_var =", d_var)
    print("in a_func e_var =", e_var)
    return b_var + 10

c_var = a_func(b_var)

print("a_var =", a_var)
print("b_var =", b_var)
print("c_var =", c_var)
print("d_var =", d_var)
```

output:

```
in a_func a_var =  15
in a_func b_var =  115
in a_func d_var =  30
in a_func e_var =  25
a_var =  10
b_var =  15
c_var =  125
d_var =

Traceback (most recent call last):
 File "C:\def2.py", line 19, in <module>
   print("d_var = ", d_var)
NameError: name 'd_var' is not defined
```

In this example the variables `a_var`, `b_var`, and `d_var` are all local variables when they are inside the function `a_func`. After the statement `return b_var + 10` is run, they all cease to exist. The variable `a_var` is automatically a local variable since it is a parameter name. The variables `b_var` and `d_var` are local variables since they appear on the left of an equals sign in the function in the statements `b_var = 100 + a_var` and `d_var = 2 * a_var` .

Inside of the function `a_var` has no value assigned to it. When the function is called with `c_var = a_func(b_var)`, 15 is assigned to `a_var` since at that point in time `b_var` is 15,

making the call to the function `a_func(15)`. This ends up setting `a_var` to 15 when it is inside of `a_func`.

As you can see, once the function finishes running, the local variables `a_var` and `b_var` that had hidden the global variables of the same name are gone. Then the statement `print("a_var = ", a_var)` prints the value `10` rather than the value `15` since the local variable that hid the global variable is gone.

Another thing to notice is the `NameError` that happens at the end. This appears since the variable `d_var` no longer exists since `a_func` finished. All the local variables are deleted when the function exits. If you want to get something from a function, then you will have to use `return something`.

One last thing to notice is that the value of `e_var` remains unchanged inside `a_func` since it is not a parameter and it never appears on the left of an equals sign inside of the function `a_func`. When a global variable is accessed inside a function it is the global variable from the outside.

Functions allow local variables that exist only inside the function and can hide other variables that are outside the function.

8.0.3 Examples

temperature2.py

```
#! /usr/bin/python
#-*-coding: utf-8 -*-
# converts temperature to Fahrenheit or Celsius

def print_options():
    print("Options:")
    print(" 'p' print options")
    print(" 'c' convert from Celsius")
    print(" 'f' convert from Fahrenheit")
    print(" 'q' quit the program")

def celsius_to_fahrenheit(c_temp):
    return 9.0 / 5.0 * c_temp + 32

def fahrenheit_to_celsius(f_temp):
    return (f_temp - 32.0) * 5.0 / 9.0

choice = "p"
while choice != "q":
    if choice == "c":
        c_temp = float(input("Celsius temperature: "))
        print("Fahrenheit:", celsius_to_fahrenheit(c_temp))
        choice = input("option: ")
    elif choice == "f":
        f_temp = float(input("Fahrenheit temperature: "))
        print("Celsius:", fahrenheit_to_celsius(f_temp))
        choice = input("option: ")
    else:
        choice = "p"    #Alternatively choice != "q": so that print
                        #when anything unexpected inputed
        print_options()
        choice = input("option: ")
```